

AI Fraud Payment Detection System - Project Explanation

Table of Contents

1. [Project Introduction](#)
 2. [Project Objective](#)
 3. [Project Architecture](#)
 4. [Technology Stack](#)
 5. [Dataset Generation](#)
 6. [Feature Engineering](#)
 7. [Unsupervised Learning Models](#)
 8. [Ensemble Risk Engine](#)
 9. [Behavioral Scoring Logic](#)
 10. [Explainability Engine](#)
 11. [Face Verification Workflow](#)
 12. [API Workflow](#)
 13. [Frontend Workflow](#)
 14. [Visualization and Analytics](#)
 15. [End-to-End Workflow](#)
 16. [Advantages of the Project](#)
 17. [Limitations](#)
 18. [Future Improvements](#)
 19. [Conclusion](#)
-

1. Project Introduction

What the Project Is

This project is an AI-powered fraud payment detection system that simulates how a modern payment monitoring platform can identify suspicious financial transactions using unsupervised machine learning. It provides a complete full-stack application with:

- A React frontend for entering transactions, viewing fraud decisions, exploring analytics, comparing models, and managing transaction history.
- A FastAPI backend for transaction simulation, model inference, history storage, analytics, and verification.
- An unsupervised machine learning pipeline based on Isolation Forest, One-Class SVM, and DBSCAN.
- A SQLite database for storing simulated transaction outcomes.
- A camera-based verification workflow for transactions that need additional identity confirmation.

The system is designed as an educational and portfolio-ready project. It demonstrates not only model prediction, but also the surrounding application architecture needed to make fraud detection understandable and usable.

What Problem It Solves

Digital payments are fast, convenient, and widely used, but they also create opportunities for fraud. A fraudster may attempt to:

- Make a high-value transaction from a stolen account.
- Use a legitimate user account from an unusual location.
- Perform payments at abnormal hours.
- Execute patterns that do not match the user's historical behavior.

Traditional rule-based fraud systems use fixed rules such as "block if amount is above 50000" or "review if transaction occurs after midnight." These rules are useful, but they are also rigid. Real fraud behavior changes over time. A fixed rule may miss new fraud strategies or incorrectly block genuine users.

This project addresses that problem by modeling normal behavior and identifying deviations from it. Instead of only asking "Does this transaction match a fixed rule?", the system asks "Does this transaction look unusual compared with known behavior patterns?"

Why Fraud Payment Detection Matters

Fraud detection is important because financial systems operate under high trust. A single weak point can affect:

- Customers, who may lose money or confidence in a service.
- Banks and payment companies, which may face financial loss and reputational damage.
- Merchants, who may receive chargebacks or delayed settlements.
- Regulators, who expect financial institutions to monitor suspicious activity.

In real-world systems, fraud detection is not only about blocking fraud. It must also avoid blocking genuine users. A system that blocks too aggressively creates poor user experience. A system that is too relaxed allows fraud. The practical challenge is balancing security with usability.

Why Unsupervised Learning Is Used

The project uses unsupervised learning because fraud labels are often difficult to obtain. In a supervised fraud classification system, every training example ideally needs a label such as "fraud" or "not fraud." In real banking environments, this is challenging because:

- Fraud may be discovered days or weeks later.
- Some suspicious transactions are never confirmed.
- Fraud patterns change quickly.
- Genuine labeled fraud examples are rare compared with normal transactions.
- Labels may be noisy, incomplete, or biased.

Unsupervised anomaly detection avoids this dependency during model training. It learns the structure of normal transaction behavior and flags transactions that deviate from that structure. In this project, synthetic anomaly labels exist for evaluation and analytics, but the unsupervised models are trained on feature patterns rather than using labels as the target.

Supervised Fraud Classification vs Unsupervised Anomaly Detection

Supervised fraud classification learns from labeled examples. A typical supervised model receives examples like:

- Transaction A: legitimate
- Transaction B: fraudulent
- Transaction C: legitimate

It learns decision boundaries that separate fraud from non-fraud. This approach can be very accurate when high-quality labels are available, but it depends heavily on labeled data.

Unsupervised anomaly detection works differently. It does not require labels during training. Instead, it learns what normal data looks like and treats rare or structurally different samples as suspicious. This is suitable when confirmed fraud labels are scarce or when the goal is to discover unusual behavior rather than classify known fraud patterns.

In this project:

- The models learn from transaction features such as amount, time, location, and user identity.
- The system scores how unusual a new transaction appears.
- A risk engine converts anomaly signals into a business decision: SAFE, REVIEW, or BLOCKED.

Real-World Inspiration Behind Behavioral Fraud Systems

Modern fraud systems often use behavioral intelligence. Instead of evaluating a transaction in isolation, they compare it with a user's normal behavior. For example:

- A user normally spends between INR 500 and INR 5000, but suddenly attempts INR 98000.
- A user usually transacts in Mumbai, Delhi, and Bengaluru, but suddenly appears in an uncommon location.
- A user usually pays during daytime, but a transaction occurs at 2:00 AM.

Real banking systems may use hundreds of signals, including device fingerprinting, IP reputation, merchant history, velocity patterns, geolocation, failed login attempts, and historical chargebacks. This project focuses on a smaller but meaningful feature set to demonstrate the core idea clearly.

2. Project Objective

Goal of the System

The goal of this system is to simulate a fraud detection pipeline that can:

1. Accept payment transaction details from a user interface.

2. Convert raw transaction data into machine-learning features.
3. Score the transaction using multiple unsupervised anomaly detection models.
4. Combine model outputs with behavioral deviation logic.
5. Generate a risk score and final classification.
6. Explain why a transaction is considered safe or suspicious.
7. Trigger camera verification for transactions requiring identity confirmation.
8. Store all transaction outcomes in a database.
9. Provide dashboards and analytics for fraud monitoring.

The project is not only a machine learning script. It is a complete application that shows how ML output can be integrated into an operational fraud decision system.

How Behavioral Anomaly Detection Works

Behavioral anomaly detection is based on the idea that users develop patterns over time. For example, a user may usually:

- Spend around a certain amount range.
- Transact during certain hours.
- Operate from a few common locations.

The system first builds a behavioral baseline from synthetic historical data. For each user, it computes:

- Amount quantiles such as low, high, and extreme amount boundaries.
- Average transaction hour and standard deviation of transaction hours.
- Common locations used by that user.

When a new transaction arrives, it is compared against this learned profile. If the transaction falls far outside the user's normal behavior, it receives additional risk.

Why Amount, Time, and Location Were Selected

The project uses amount, transaction time, location, and user ID because these are intuitive and important signals in payment fraud detection.

Amount is important because fraud attempts often involve extracting maximum value quickly. A sudden transaction much higher than a user's usual payment behavior can be suspicious. The project also considers unusually low amounts because fraudsters sometimes test stolen credentials with small transactions before attempting larger ones.

Time is important because users often have regular activity windows. A person who usually transacts in the evening may appear suspicious if a payment occurs at an unusual hour. The system does not simply treat "night" as suspicious for everyone; it compares the transaction hour with the user's own profile.

Location is important because unexpected location changes can indicate account takeover, stolen credentials, or unusual access. In this project, locations are represented using a fixed city index. A transaction from outside the user's common locations adds risk.

User ID is included because behavior is user-specific. Without user identity, the system can only learn global patterns. With user ID, the system can build individualized expectations.

How the System Learns Normal Behavior Patterns

The system generates a synthetic behavioral dataset containing many users and transactions. Each user receives:

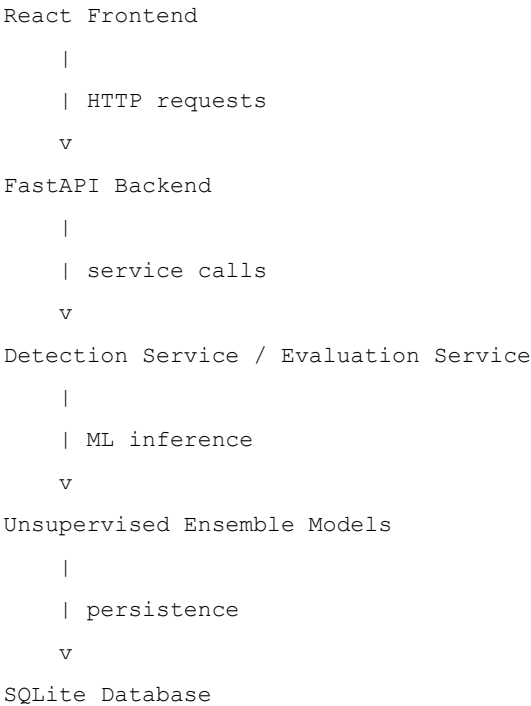
- A typical spending range.
- A preferred transaction hour.
- A secondary transaction hour.
- Three preferred locations.

Most transactions are generated from these normal patterns. A smaller percentage of anomalies are injected by increasing amounts, changing transaction hours, or moving transactions to uncommon locations. However, the unsupervised models do not use anomaly labels for training. The labels are mainly used later for evaluation metrics such as AUC and average precision.

3. Project Architecture

High-Level Architecture

The application follows a full-stack architecture:



The major layers are:

- **Frontend:** provides user interaction and visual analytics.
- **API layer:** exposes endpoints for transaction simulation, verification, history, dashboard, and evaluation.
- **ML pipeline:** preprocesses data and scores transactions.
- **Risk engine:** combines model outputs and behavioral deviations into a final decision.

- Database: persists transaction records and verified state.
- Visualization system: presents risk trends, distributions, model comparisons, and transaction history.

Frontend Layer

The frontend is built with React and Tailwind CSS. It provides several pages:

- Dashboard: summary cards, risk distribution, fraud risk trend, and live activity feed.
- Payment Simulator: form for entering transactions and viewing AI decision outcomes.
- Fraud Analytics: anomaly charts and risk distributions.
- Model Comparison: model-level anomaly signal comparison and evaluation metrics.
- Transaction History: stored transactions, delete action, and verification actions.

The frontend communicates with the backend through the API client in `frontend/src/api/client.js`.

Backend Layer

The backend is built with FastAPI. It handles:

- Request validation through Pydantic schemas.
- API routing.
- Calling the detection service.
- Storing transactions in SQLite.
- Returning structured JSON responses.
- Updating transactions after verification.
- Deleting transactions from history.

The main router is defined in `backend/app/api/routes.py`.

ML Pipeline

The ML pipeline is responsible for:

1. Generating synthetic behavioral data.
2. Building numerical feature matrices.
3. Scaling features.
4. Training unsupervised models.
5. Scoring new transactions.
6. Normalizing model outputs.

The ensemble is implemented in `backend/app/ml/models.py`.

Database Layer

The project uses SQLite through SQLAlchemy. The database stores:

- User ID
- Amount

- Transaction time
- Location
- Anomaly score
- Risk score
- Individual model scores
- Classification
- Explanation
- Face verification requirement
- Created timestamp

SQLite is suitable for this project because it is simple, file-based, and easy to run locally without additional database setup.

API Layer

The API layer exposes endpoints for:

- Health check
- Transaction simulation
- Face verification
- Transaction deletion
- Transaction history
- Dashboard analytics
- Model evaluation analytics

These endpoints allow the frontend to operate as a real application rather than a static demo.

Risk Engine

The risk engine combines two types of intelligence:

1. Model-based anomaly signals from Isolation Forest, One-Class SVM, and DBSCAN.
2. User-specific behavioral deviation logic based on amount, time, and location.

This combination makes the decision more realistic. A model may detect an unusual feature pattern, while behavioral rules provide interpretable domain context.

Visualization System

The visualization system uses Recharts to present:

- Area charts for risk trends.
- Pie charts for risk distribution and model weights.
- Bar charts for model anomaly signals.
- Analytics charts for anomaly distribution and location risk.

These visualizations are important because fraud systems are usually monitored by analysts. A risk score alone is less useful than a dashboard showing trends and context.

Complete Transaction Workflow

The complete workflow is:

```
User enters transaction
->
Frontend sends transaction to FastAPI
->
Backend validates request
->
Feature engineering converts input to numeric vector
->
Feature pipeline scales vector
->
Isolation Forest, One-Class SVM, and DBSCAN score the transaction
->
Scores are normalized
->
Weighted ensemble score is calculated
->
User behavioral deviation score is calculated
->
Final risk score is generated
->
Classification is assigned
->
Explanation is generated
->
Transaction is stored in SQLite
->
Frontend displays decision and charts
->
If REVIEW or BLOCKED, camera verification can mark it SAFE
```

4. Technology Stack

React

React is used to build the frontend interface. It is component-based, which makes it suitable for pages such as Dashboard, Payment Simulator, Model Comparison, and Transaction History. React state is used to store the current page, latest decision, dashboard data, history records, and evaluation data.

React was selected because:

- It supports dynamic interfaces.
- It is widely used in modern web applications.
- It makes it easy to update UI after API calls.
- It works well with reusable page and component structures.

Tailwind CSS

Tailwind CSS is used for styling. It allows the project to define layouts, spacing, colors, borders, and responsive behavior directly in class names.

Tailwind was selected because:

- It enables fast UI development.
- It keeps styling consistent.
- It supports responsive design easily.
- It avoids writing large custom CSS files.

The project uses a dark cyber-style visual theme with glass-like panels, cyan accents, and status colors for SAFE, REVIEW, and BLOCKED.

Framer Motion

Framer Motion is used for animations. It powers smooth page transitions, modal animations, progress animations, and subtle UI motion.

It was selected because:

- It integrates naturally with React.
- It improves user experience.
- It makes state changes feel smoother.
- It helps the verification modal and analysis flow feel interactive.

Recharts

Recharts is used for charts and analytics. The project uses charts such as:

- PieChart
- AreaChart
- BarChart
- ComposedChart

Recharts was selected because:

- It is React-friendly.
- It provides responsive containers.
- It supports common chart types.
- It is suitable for dashboards and analytics views.

FastAPI

FastAPI is used for the backend API. It is a modern Python web framework designed for high performance and clean API development.

FastAPI was selected because:

- It supports automatic request validation.
- It integrates well with Pydantic models.
- It is easy to structure APIs.
- It provides clear JSON responses.
- It is suitable for ML-backed services.

SQLite

SQLite is used as the database. It stores transaction records in a local file called `fraud_detection.db`.

SQLite was selected because:

- It requires no separate database server.
- It is easy to run locally.
- It is sufficient for a project prototype.
- It works well with SQLAlchemy.

Scikit-learn

Scikit-learn is used for machine learning. It provides:

- IsolationForest
- OneClassSVM
- DBSCAN
- NearestNeighbors
- PCA
- Scaling pipelines
- Evaluation metrics such as ROC AUC and average precision

Scikit-learn was selected because:

- It is reliable and widely used.
- It has strong unsupervised learning support.
- It provides preprocessing and evaluation tools.
- It is beginner-friendly while still being technically powerful.

Python

Python is used for backend services and machine learning. It is suitable because:

- It has a strong ML ecosystem.

- It is easy to read and explain.
 - It works well with FastAPI, NumPy, SQLAlchemy, and Scikit-learn.
-

5. Dataset Generation

Why a Synthetic Dataset Was Created

Real payment data is private, sensitive, and difficult to access. Banking datasets contain personally identifiable information, transaction histories, and confidential fraud patterns. For ethical and practical reasons, this project uses a synthetic dataset.

A synthetic dataset allows the project to:

- Simulate realistic user behavior.
- Control anomaly injection.
- Train unsupervised models without real customer data.
- Evaluate model behavior using known synthetic anomaly labels.
- Make the project reproducible.

How Realistic Behavior Was Simulated

The synthetic data generator creates a population of users. Each user receives a behavioral profile:

- Mean transaction amount
- Standard deviation of amount
- Preferred transaction hour
- Secondary transaction hour
- Three preferred locations

Most transactions follow these profiles. This creates normal behavior clusters. For example, one user may usually spend around INR 2000 in Mumbai, Delhi, and Bengaluru around 7 PM. Another user may usually spend around INR 7000 in Kolkata, Pune, and Jaipur during the morning.

How User Profiles Are Generated

The generator creates 900 users by default. For each user:

- `user_mean_amount` is sampled from a range of INR 450 to INR 8500.
- `user_std_amount` is based on a percentage of the user's mean amount.
- `user_preferred_hour` is selected between 7 and 22.
- `user_secondary_hour` is derived from the preferred hour.
- Three preferred cities are selected from a fixed city pool.

These profiles make behavior user-specific rather than globally identical.

How Normal Behavior Patterns Are Generated

For each transaction:

1. A user is selected.
2. The amount is sampled around that user's mean amount.
3. The hour is usually close to the user's preferred hour.
4. Sometimes the hour is close to the user's secondary hour.
5. The location is selected from one of the user's preferred locations.

The result is a dataset where most transactions are not random. They follow structured patterns that can be learned by anomaly detection algorithms.

How Anomalies Are Injected

The generator marks around 7.5 percent of transactions as anomalies. These synthetic anomalies are created in three main ways:

- Amount anomalies: the amount is multiplied by a large factor.
- Time anomalies: the transaction hour is moved to unusual hours such as midnight or early morning.
- Location anomalies: the location is changed to a city outside the user's preferred location set.

These injected anomalies represent common fraud signals:

- Sudden high-value activity
- Unusual activity time
- Geographic inconsistency

Why Labels Are Not Used During Training

Although the synthetic dataset includes `anomaly_label`, the unsupervised models are not trained as supervised classifiers.

The labels are not used as target values during model fitting.

The labels are mainly used for:

- Evaluation metrics
- Model comparison
- Analytics summaries

This design is important because the project is about unsupervised anomaly detection. The model should learn structure from the data itself rather than being directly told which samples are fraudulent.

Generated Fields

`transaction_id`

This is a synthetic identifier such as `TXN-0000001`. It gives each generated transaction a unique reference.

`user_id`

This identifies the simulated user. User ID is important because the system builds user-specific behavioral profiles.

amount

This is the payment amount. It is one of the most important fraud detection signals because abnormal amounts can indicate suspicious behavior.

transaction_time

This is a string in `HH:MM` format. It represents the time of the transaction and is later converted into hour-based features.

location

This is the city where the transaction is assumed to occur. The project uses a fixed city pool including Mumbai, Delhi, Bengaluru, Hyderabad, Chennai, Kolkata, Pune, Ahmedabad, Jaipur, Lucknow, Kochi, and Surat.

6. Feature Engineering

Feature engineering converts raw transaction data into numerical form so machine learning models can process it.

Amount Transformation

The amount is transformed using:

```
log1p(amount)
```

This means the system computes the natural logarithm of $1 + \text{amount}$. The reason is that transaction amounts can vary widely. A difference between INR 100 and INR 1000 is not the same as a difference between INR 100000 and INR 100900.

Log transformation reduces the impact of very large values while preserving the order of amounts.

User ID Transformation

The user ID is transformed using:

```
log1p(user_id)
```

This gives the model some user-specific signal without making raw user IDs dominate the feature space.

Time Encoding

Time is not treated as a simple number from 0 to 23. If raw hour values were used directly, the model would think hour 23 and hour 0 are far apart. In reality, 11 PM and midnight are adjacent.

To solve this, the project uses cyclical encoding.

Cyclical Encoding Using Sin and Cos

The hour is converted into an angle:

```
angle = 2 * pi * (hour / 24)
```

Then two features are created:

```
hour_sin = sin(angle)
hour_cos = cos(angle)
```

This represents time as a point on a circle. It correctly captures that hour 23 and hour 0 are close.

Location Encoding

Each known city is mapped to a number:

```
mumbai -> 1
delhi -> 2
bengaluru -> 3
...
surat -> 12
```

Unknown locations are mapped to 0. This gives the models a simple numerical representation of location.

In a production system, a more advanced representation could be used, such as latitude/longitude, geo-IP data, device region, or learned embeddings.

Scaling and Normalization

The feature pipeline uses:

1. RobustScaler
2. MinMaxScaler

RobustScaler is useful because it is less sensitive to outliers than standard scaling. This matters in fraud detection because extreme values may exist in the data.

MinMaxScaler then converts features into a 0–1 range. This helps models such as One-Class SVM and DBSCAN, which are sensitive to feature scale.

Why Preprocessing Is Important for Anomaly Detection

Anomaly detection models are highly affected by feature representation. If one feature has a much larger numerical range than another, it can dominate distance calculations or decision boundaries.

For example, raw amount values may be in thousands while hour values are only between 0 and 23. Without scaling, amount could overpower time and location. Preprocessing ensures that each feature contributes more fairly.

7. Unsupervised Learning Models

The project uses three unsupervised models. Each model detects anomalies in a different way. Combining them gives a broader and more reliable fraud signal.

A. Isolation Forest

How Isolation Forest Works

Isolation Forest is based on the idea that anomalies are easier to isolate than normal points. It builds many random decision trees. Each tree randomly selects a feature and a split value. Points that require fewer splits to isolate are considered more anomalous.

For example, if most users spend between INR 1000 and INR 7000, a transaction of INR 200000 may be isolated quickly because it lies far from the majority.

Why It Is Useful for Anomaly Detection

Isolation Forest is useful because:

- It handles high-dimensional data reasonably well.
- It is efficient for large datasets.
- It does not require labeled fraud examples.
- It can detect global outliers.

Why It Was Selected

It was selected because payment fraud often contains outliers in amount, time, or location. Isolation Forest is a strong baseline for detecting such outliers.

Strengths

- Fast and scalable.
- Good for extreme anomalies.
- Works well without labels.
- Suitable for numerical feature vectors.

Weaknesses

- It may not capture all local behavioral patterns.
- It can be sensitive to contamination settings.
- Its score is not a true probability.

Role in This Project

Isolation Forest has the highest ensemble weight at 45 percent. It acts as the primary global anomaly detector.

B. One-Class SVM

How One-Class SVM Works

One-Class SVM learns a boundary around normal data. It attempts to separate normal observations from the rest of the feature space. Transactions outside this learned boundary are treated as suspicious.

The project uses an RBF kernel, which allows non-linear boundaries. This is useful because normal behavior may not be linearly separable.

Why Boundary Learning Is Useful for Behavioral Detection

Fraud detection often involves determining whether a transaction lies within a known region of normal behavior. One-Class SVM is designed for this type of problem. It can learn a flexible boundary around normal transaction patterns.

Strengths

- Learns non-linear boundaries.
- Useful when normal behavior forms complex regions.
- Does not require fraud labels.

Weaknesses

- Can be slower on large datasets.
- Sensitive to parameters such as `nu` and `gamma`.
- Scores are not probabilities.

Role in This Project

One-Class SVM receives 35 percent weight in the ensemble. It complements Isolation Forest by focusing on boundary-based abnormality.

C. DBSCAN

How DBSCAN Works

DBSCAN is a density-based clustering algorithm. It groups points that are close together and have enough nearby neighbors. Points that do not belong to dense regions are marked as noise.

The core idea is:

- Dense regions represent normal clusters.
- Sparse points may represent anomalies.

Clustering and Density-Based Anomaly Detection

In fraud detection, normal behavior often forms dense clusters. For example, many normal transactions may occur in similar amount ranges, times, and locations. A transaction far from these clusters may be suspicious.

DBSCAN identifies cluster structure without needing a predefined number of clusters.

Noise Points

DBSCAN labels points that do not belong to any cluster as -1 . These are considered noise points. In anomaly detection, noise points can be interpreted as suspicious.

Why DBSCAN Is Useful

DBSCAN is useful because:

- It can detect arbitrary-shaped clusters.
- It does not require specifying the number of clusters.
- It naturally identifies noise.

Strengths

- Good for density-based anomaly detection.
- Finds cluster structure.
- Identifies noise directly.

Weaknesses

- Sensitive to `eps` and `min_samples`.
- Can struggle when clusters have different densities.
- In this project, DBSCAN has limited discriminative power compared with the other models.

Role in This Project

DBSCAN receives 20 percent ensemble weight. The implementation uses nearest-neighbor distance to DBSCAN core samples as the anomaly signal for new transactions.

8. Ensemble Risk Engine

Why an Ensemble Is Used

No single anomaly detection model is perfect. Different algorithms detect different types of unusual behavior:

- Isolation Forest detects easy-to-isolate outliers.
- One-Class SVM detects boundary violations.
- DBSCAN detects distance from dense clusters.

An ensemble combines these perspectives. This makes the system more robust than relying on one model.

How Outputs Are Combined

Each model produces a raw anomaly-related score. These raw values are not directly comparable, so they are normalized to a $0-1$ anomaly signal:

- 0 means less anomalous.

- 1 means highly anomalous relative to the reference range.

The model scores are then combined using weighted scoring:

```
weighted_model_score =  
    0.45 * isolation_forest_score  
+ 0.35 * one_class_svm_score  
+ 0.20 * dbscan_score
```

Normalization

Isolation Forest and One-Class SVM use decision functions where lower values are more suspicious. Therefore, their scores are inverted and normalized using reference quantiles from the training data.

DBSCAN uses nearest-neighbor distance to core samples. Larger distance means more unusual, so it is normalized directly.

The current implementation clips normalized values between 0.0 and 1.0. In the frontend, displayed model anomaly signals are capped at 0.95 to avoid visually implying perfect certainty. Internally, the backend still uses the actual normalized values for scoring.

Behavioral Deviation Contribution

The project also calculates `user_deviation`, based on:

- Amount higher than the user's 95th percentile.
- Amount lower than the user's 5th percentile.
- Transaction time far from the user's mean hour.
- Location outside the user's common locations.

The final ensemble score is:

```
final_score =  
    0.78 * weighted_model_score  
+ 0.22 * user_deviation
```

This means model outputs are the main factor, but user-specific behavior also influences the final result.

Risk Score

The final score is multiplied by 100:

```
risk_score = final_score * 100
```

The risk score is stored and shown in the frontend.

Current Risk Classification Logic

The current backend implementation uses:

0.00 - 54.99	SAFE
55.00 - 74.99	REVIEW
75.00 - 100.00	BLOCKED

This classification was calibrated so normal behavioral transactions can realistically appear as SAFE, while suspicious transactions can be reviewed or blocked.

Some analytics labels in the project still use older display bucket names such as `safe_0_35`, `review_36_70`, and `blocked_71_100` for histogram grouping. However, the actual live transaction decision logic in `DetectionService` currently uses the thresholds above: `<55 SAFE`, `55-74.99 REVIEW`, and `>=75 BLOCKED`.

Weight Distribution

The weights are:

- Isolation Forest: 45 percent
- One-Class SVM: 35 percent
- DBSCAN: 20 percent

Isolation Forest has the highest weight because it is generally strong for outlier detection. One-Class SVM receives a substantial weight because it learns behavioral boundaries. DBSCAN receives a smaller but meaningful weight because it adds density-based perspective, even though it may be less sensitive in this implementation.

9. Behavioral Scoring Logic

How User Behavior Patterns Are Learned

After synthetic data is generated, the backend builds a profile for each user:

- `amount_q005`: extremely low amount boundary.
- `amount_q05`: low amount boundary.
- `amount_q95`: high amount boundary.
- `amount_q995`: extremely high amount boundary.
- `hour_mean`: average transaction hour.
- `hour_std`: standard deviation of transaction hour.
- `common_locations`: frequently used locations.

These profiles are stored in memory inside `DetectionService`.

How Deviations Are Detected

When a transaction is evaluated:

- If amount is above the user's 95th percentile, risk increases.
- If amount is below the user's 5th percentile, risk increases.
- If transaction time is far from the user's typical time range, risk increases.
- If location is outside common locations, risk increases.

This user deviation score is clipped between 0 and 1.

Examples of Suspicious Behavior

Amount Higher Than Usual

A user normally spends below INR 8000, but suddenly attempts INR 98000. The explanation engine may produce:

```
Risk factors detected: amount higher than usual, unusual location, high anomaly confidence.
```

Amount Lower Than Usual

Very low amounts can be suspicious because attackers may test a stolen account with a small transaction. The explanation engine may produce:

```
Risk factors detected: amount lower than usual, high anomaly confidence.
```

Abnormal Transaction Time

A user normally transacts around 8 PM, but a transaction occurs at 2 AM. The explanation engine may include:

```
unusual transaction time
```

Unusual Location

A user normally transacts in Kolkata, Jaipur, and Ahmedabad, but a payment appears from Singapore. The system may include:

```
unusual location
```

How Anomaly Confidence Contributes

The explanation engine adds `high anomaly confidence` when at least two model scores are greater than or equal to 0.85. This means multiple models strongly agree that the transaction is unusual.

The phrase should be understood as an anomaly signal, not a true statistical probability. Unsupervised models do not produce calibrated fraud probabilities.

10. Explainability Engine

What Explainability Means

Explainability means the system provides human-readable reasons for its decisions. Instead of only displaying:

```
BLOCKED
```

the system also explains why:

```
Risk factors detected: amount higher than usual, unusual location, high anomaly confidence.
```

Why Explainability Is Important

Explainability is important in fraud systems because:

- Analysts need to understand why a transaction was flagged.
- Users may need support if a transaction is blocked.
- Reviewers and auditors need decision transparency.
- Blind model scores are difficult to trust.

How Explanations Are Generated

The explanation engine checks:

- Whether amount is extremely high or low for the user.
- Whether the hour is far from normal user activity.
- Whether location is outside common locations.
- Whether multiple ML models show high anomaly signals.

If no suspicious factors are found, it returns:

```
Transaction pattern aligns with normal user behavior.
```

If suspicious factors exist, it returns:

```
Risk factors detected: <factor 1>, <factor 2>, <factor 3>.
```

Example Explanations

Safe example:

Transaction pattern aligns with normal user behavior.

Review example:

Risk factors detected: unusual location.

Blocked example:

Risk factors detected: amount higher than usual, unusual location, high anomaly confidence.

Verified example:

Camera verification completed. Transaction verified and marked safe.

11. Face Verification Workflow

Why Verification Is Triggered

The system triggers verification for transactions classified as REVIEW or BLOCKED. These are transactions that are not confidently safe and therefore need an extra identity check.

In real financial systems, this is called step-up authentication. A transaction may require:

- Face verification
- OTP verification
- Device confirmation
- Banking app approval
- Security question

This project uses camera-based verification to simulate a modern identity confirmation flow.

How Review and Blocked Transactions Are Handled

For REVIEW or BLOCKED transactions:

1. The frontend displays an `Open Camera Verification` or `Verify` action.
2. The browser requests camera access.
3. The user aligns their face in the modal.
4. The user captures an image.
5. The frontend calls the backend verification endpoint.
6. The backend simulates successful face verification.
7. The transaction is updated in the database:
 - classification becomes `SAFE`

- `face_verification_required` becomes false
 - `explanation` is updated
8. The frontend refreshes dashboard and history.

Payment Simulator Verification

In the Payment Simulator page, the camera modal appears when the latest transaction is REVIEW or BLOCKED. After capture, the decision panel updates based on the returned transaction.

Transaction History Verification

In the Transaction History page, REVIEW and BLOCKED status badges are clickable. The user can also click a `Verify` button. This opens the same style of camera modal. After successful verification, the row becomes SAFE when the history refreshes.

Why Simulation Was Used Instead of Real Facial Recognition

Real facial recognition requires:

- Secure biometric storage
- Face matching algorithms
- Liveness detection
- Privacy compliance
- Consent and encryption
- Anti-spoofing protection

These are complex and sensitive topics. For this academic project, the goal is to demonstrate the workflow, not to build a production biometric system. Therefore, the backend uses a simulated verification service that returns a successful match with confidence `0.96`.

12. API Workflow

Base API Prefix

All backend routes are mounted under:

```
/api/v1
```

The frontend API client uses:

```
http://127.0.0.1:8000/api/v1
```

GET /health

Purpose:

- Checks whether the backend is running.

Response:

```
{  
  "status": "ok"  
}
```

POST /transactions/simulate

Purpose:

- Accepts transaction input.
- Runs ML fraud detection.
- Stores the transaction.
- Returns the decision.

Request:

```
{  
  "user_id": 318,  
  "amount": 6500,  
  "transaction_time": "20:45",  
  "location": "Kolkata"  
}
```

Response includes:

- transaction ID
- user ID
- amount
- time
- location
- anomaly score
- ensemble score
- risk score
- classification
- explanation
- face verification requirement
- model scores
- created timestamp

POST /transactions/

Purpose:

- Performs simulated camera verification for REVIEW or BLOCKED transactions.
- Updates successful transactions to SAFE.

Backend behavior:

1. Finds the transaction.
2. Rejects the request if the transaction does not exist.
3. Rejects the request if the transaction is not REVIEW or BLOCKED.
4. Runs simulated verification.
5. If approved, updates the database row.
6. Returns verification result and updated transaction.

Response example:

```
{
  "approved": true,
  "confidence": "0.96",
  "message": "Face match succeeded",
  "transaction": {
    "classification": "SAFE",
    "face_verification_required": false
  }
}
```

DELETE /transactions/

Purpose:

- Deletes a transaction from history.

Response:

```
{
  "deleted": true
}
```

GET /transactions/history

Purpose:

- Returns recent transactions from the database.

Query parameter:

```
limit=100
```

The frontend uses this to populate Transaction History, Dashboard live feed, and latest transaction state.

GET /analytics/dashboard

Purpose:

- Returns summary statistics:
 - total transactions
 - blocked count
 - review count
 - safe count
 - average risk score

This powers the dashboard cards.

GET /analytics/evaluation

Purpose:

- Returns model evaluation analytics:
 - dataset size
 - anomaly rate
 - risk distribution
 - model comparison metrics
 - PCA points
 - DBSCAN cluster data

This powers Fraud Analytics and Model Comparison pages.

13. Frontend Workflow

Dashboard Flow

The Dashboard page loads:

- Transaction summary cards.
- Risk distribution chart.
- Fraud risk trend chart.
- Live activity feed.

The live activity feed is scrollable and shows recent activity. This makes the dashboard useful for quick monitoring.

Payment Simulation Flow

The Payment Simulator page allows the user to enter:

- User ID
- Amount
- Transaction time
- Location

It also provides preset buttons such as Routine, High value, and New city. After submission:

1. The frontend sends the transaction to the backend.
2. The backend evaluates and stores the transaction.
3. The frontend displays classification, risk score, model scores, and explanation.
4. If the transaction is REVIEW or BLOCKED, verification is available.

Analytics Flow

Fraud Analytics displays:

- Hourly fraud/risk activity.
- Anomaly distribution.
- Location anomaly chart.

These charts help explain how risk behaves across generated or simulated transactions.

Model Comparison Flow

Model Comparison displays:

- Model anomaly signals for the latest transaction.
- Ensemble weight visualization.
- Interactive model switching.
- AUC and average precision for the selected model.

The frontend caps displayed model anomaly signals at 0.95 so high-risk values do not visually imply perfect certainty.

Transaction History Flow

Transaction History displays all recent stored transactions. Each row shows:

- User
- Amount
- Time
- Location
- Risk
- Status
- Actions

Users can:

- Delete any transaction.
- Click REVIEW or BLOCKED status badges to verify.
- Click the Verify button for REVIEW or BLOCKED rows.

After verification, the row becomes SAFE because the backend updates the stored transaction.

Verification Modal Flow

The verification modal:

1. Opens camera access with `navigator.mediaDevices.getUserMedia`.
 2. Shows the live video preview.
 3. Captures a frame into a canvas.
 4. Converts it into an image preview.
 5. Stops camera tracks.
 6. Calls the backend face verification endpoint.
 7. Shows a Verified overlay.
 8. Refreshes application data.
-

14. Visualization and Analytics

PCA Visualization

PCA stands for Principal Component Analysis. It reduces high-dimensional data into two dimensions so patterns can be visualized. The backend creates PCA points from scaled transaction features.

The purpose is to show how transactions are distributed in feature space. Anomalies may appear away from dense normal regions.

Anomaly Charts

The analytics charts show risk and anomaly behavior over transactions. This helps reviewers understand whether risk is increasing, decreasing, or clustering around certain areas.

Cluster Visualizations

DBSCAN cluster data is generated by the backend. DBSCAN labels dense groups and noise points. Noise points represent transactions that do not fit into normal dense clusters.

Fraud Analytics Dashboard

The Fraud Analytics page includes:

- Hourly Fraud Activity
- Anomaly Distribution Heatmap
- Location Anomaly Chart

These charts make fraud monitoring easier to present and explain.

Model Comparison Analytics

The Model Comparison page includes:

- Bar chart of model anomaly signals.
- Pie chart of ensemble weights.
- Interactive model switching.
- Per-model AUC and average precision.

This helps explain which model contributes what kind of signal.

15. End-to-End Workflow

Example: High-Risk Payment

Suppose a user enters:

```
User ID: 204
Amount: 98000
Time: 01:15
Location: Dubai
```

Step 1: Frontend Input

The user fills the Payment Simulator form and submits it.

Step 2: API Request

The frontend sends a POST request to:

```
/api/v1/transactions/simulate
```

Step 3: Request Validation

FastAPI validates:

- user_id is greater than 0
- amount is greater than 0
- transaction_time exists
- location exists

Step 4: Feature Engineering

The backend converts the transaction into features:

- log amount
- hour sine
- hour cosine
- location number
- log user ID

Step 5: Scaling

The feature vector is transformed using the trained RobustScaler and MinMaxScaler pipeline.

Step 6: Model Inference

The scaled vector is passed into:

- Isolation Forest
- One-Class SVM
- DBSCAN nearest-core distance scoring

Each model returns an anomaly signal.

Step 7: Ensemble Scoring

The model scores are combined:

```
0.45 * Isolation Forest
0.35 * One-Class SVM
0.20 * DBSCAN
```

Then behavioral deviation is added:

```
0.78 * model score + 0.22 * user deviation
```

Step 8: Risk Score Generation

The final score is multiplied by 100 to generate a risk score.

Step 9: Classification

The backend classifies the transaction using current thresholds:

<55	SAFE
55-74.99	REVIEW
>=75	BLOCKED

Step 10: Explainability

The backend generates reasons such as:

Risk factors detected: amount higher than usual, unusual location, high anomaly confidence.

Step 11: Storage

The transaction is stored in SQLite with its scores, classification, explanation, and verification requirement.

Step 12: Frontend Display

The frontend displays:

- Classification
- Risk score
- Dynamic risk meter
- Model scores
- Explanation
- Verification button if needed

Step 13: Verification If Needed

If the transaction is REVIEW or BLOCKED:

1. The user opens camera verification.
2. The user captures an image.
3. The backend simulates verification.
4. The transaction becomes SAFE.
5. History and dashboard refresh.

16. Advantages of the Project

Real-World Relevance

Fraud detection is a real and important problem in banking, fintech, e-commerce, and digital wallets. This project demonstrates a realistic fraud decision flow from transaction input to final risk outcome.

Scalability of Design

The architecture separates frontend, backend, ML logic, and database. This separation makes the project easier to extend. For example, the SQLite database could be replaced with PostgreSQL, or the synthetic data generator could be replaced with a real streaming source.

Unsupervised Learning Advantages

The project does not depend on labeled fraud data for training. This is useful because fraud labels are often rare, delayed, or incomplete.

Explainability

The project does not only output a score. It explains the reason for suspicious classification. This improves trust and makes it easier to present in a viva or review.

Behavioral Intelligence

The system learns user-specific behavior. This is more advanced than global rules because it considers what is normal for a specific user.

End-to-End Demonstration

The project demonstrates:

- Data generation
- Feature engineering
- ML training
- Inference
- Backend APIs
- Database persistence
- Frontend visualization
- Camera verification
- Transaction history management

This makes it strong as both an academic project and a portfolio project.

17. Limitations

Synthetic Data

The dataset is synthetic. Although it is designed to be realistic, it cannot fully capture real financial behavior. Real datasets contain more complexity, noise, seasonality, merchant categories, device fingerprints, account age, and network signals.

Simulated Verification

The face verification flow is simulated. The browser captures an image, but the backend does not perform real biometric matching. It returns a successful verification result for demonstration purposes.

Limited Features

The current feature set includes:

- Amount
- Time
- Location
- User ID

Real fraud systems may include many more features:

- Merchant category
- Device ID
- IP address
- Velocity checks
- Login history
- Transaction frequency
- Failed authentication attempts
- Card-not-present indicators

No Real Banking Integration

The project does not connect to real payment rails, bank APIs, or merchant systems. It is a simulation environment.

Score Calibration

The model scores are normalized anomaly signals, not true fraud probabilities. A score of `1.0` internally means the value reached the upper normalized anomaly boundary, not that the model is 100 percent certain. The frontend caps displayed model scores at `0.95` to avoid misleading interpretation.

DBSCAN Sensitivity

DBSCAN depends heavily on parameters like `eps` and `min_samples`. Its usefulness can vary based on data distribution.

In-Memory Model Initialization

The ML models are trained when the backend service initializes. This is acceptable for a project prototype, but production systems usually save trained models and load them from model storage.

18. Future Improvements

Real-Time Fraud Streams

The system could be extended to process transactions in real time using streaming tools such as Kafka, Redis Streams, or cloud messaging systems.

Deep Learning

Deep learning models could be added, such as:

- Autoencoders for anomaly detection
- LSTM/GRU models for sequence-based user behavior
- Graph neural networks for account and merchant networks

Geo-IP Intelligence

Location could be improved using:

- Latitude and longitude
- Geo-IP lookup
- Distance from usual location
- Country risk score
- Impossible travel detection

Real Facial Recognition

The verification workflow could be upgraded with:

- Face embeddings
- Liveness detection
- Anti-spoofing checks
- Secure biometric templates
- Consent and privacy controls

Adaptive Learning

The system could learn from verified outcomes. For example, if a reviewed transaction is verified as safe, future similar behavior could become less suspicious for that user.

Online Retraining

Instead of training only at startup, the system could periodically retrain or update models as new transactions arrive.

Better Evaluation

The project could add:

- Confusion matrix
- Precision-recall curves
- ROC curves

- Calibration plots
- Threshold tuning interface

Additional Risk Signals

Future versions could include:

- Merchant category
- Device fingerprint
- IP reputation
- Login location
- Transaction velocity
- Failed OTP attempts
- Account age
- Beneficiary history

Production Database

SQLite could be replaced with PostgreSQL or MySQL for multi-user production use.

Authentication and Authorization

The app currently does not include user login or role-based access control. A production fraud dashboard should include secure authentication.

19. Conclusion

This project successfully demonstrates a complete AI-based fraud payment detection system using unsupervised anomaly detection. It goes beyond a simple model notebook by integrating machine learning with a real backend, database persistence, frontend dashboards, transaction history, explainability, and camera-based verification simulation.

The system learns behavioral transaction patterns from synthetic data and uses three unsupervised models: Isolation Forest, One-Class SVM, and DBSCAN. Their outputs are normalized and combined through a weighted ensemble risk engine. The backend also compares each transaction against user-specific behavioral profiles to detect unusual amount, time, and location patterns.

The frontend makes the system understandable through interactive dashboards, model comparison charts, fraud analytics, and detailed transaction history. REVIEW and BLOCKED transactions can be verified using a camera-based workflow, after which the backend updates them to SAFE.

Academically, the project is valuable because it explains and demonstrates unsupervised learning, anomaly detection, feature engineering, ensemble scoring, explainability, and full-stack ML deployment. Practically, it reflects real concepts used in fraud monitoring systems, even though it uses synthetic data and simulated verification.

Overall, the project provides a strong foundation for understanding how intelligent fraud detection systems are designed, implemented, explained, and presented.

